Projeto da aula 1: catalogação automática de produtos de moda por imagem

MBA FIAP | Deep Learning e Redes Neurais Modernas | Profa. Juliana Scudilio

Dataset: **Fashion Product Images (Small)**, Kaggle, autor Param Aggarwal. `https://www.kaggle.com/datasets/paramaggarwal/fashion-product-images-small`

Entrega em grupo (os mesmos grupos do case em sala). Prazo sugerido: antes da aula 3.

---

1. O problema de negócio

Um e-commerce de moda recebe milhares de fotos de produtos por semana, vindas de centenas de fornecedores. Cada produto precisa entrar no catálogo com categoria, tipo de peça e cor, para aparecer na busca e nos filtros do site. Hoje isso é feito por uma equipe que olha foto por foto. Erros de catalogação têm custo direto: um tênis cadastrado como sandália não aparece para quem busca tênis, e uma camiseta cadastrada em "Acessórios" some do filtro de roupas.

A pergunta do projeto: **dá para um modelo preencher esses campos a partir da foto, e em que parte do catálogo ele pode decidir sozinho?**

2. O dataset

- Cerca de 44 mil produtos, cada um com uma foto pequena (por volta de 60 x 80 pixels) na pasta `images/`, nomeada pelo `id` do produto (`images/15970.jpg`).
- O arquivo `styles.csv` tem uma linha por produto com as colunas: `id`, `gender`, `masterCategory`, `subCategory`, `articleType`, `baseColour`, `season`, `year`, `usage`, `productDisplayName`.
- Os rótulos que interessam ao projeto:
  - `masterCategory`: 7 valores (Apparel, Accessories, Footwear, Personal Care e outros). É o nível mais fácil.
  - `subCategory`: 45 valores (Topwear, Shoes, Bags, Watches…).
  - `articleType`: 141 valores (Tshirts, Casual Shoes, Handbags…). Cauda longa: poucas classes concentram a maioria dos produtos e muitas classes têm poucas dezenas de exemplos.
  - `baseColour`: 46 cores.

Avisos práticos:

- Nem todo `id` do CSV tem imagem na pasta, e vice-versa. Cruze as duas listas e descarte o que não casa. Diga no relatório quantos ficaram de fora.
- Algumas linhas do CSV têm vírgulas a mais no nome do produto. Leia com `pd.read_csv("styles.csv", on_bad_lines="skip")` e registre quantas linhas foram puladas.
- As fotos são pequenas e com fundo branco, parecidas entre si. Isso facilita o treino e vai mascarar problemas que fotos reais de fornecedor teriam. Discuta isso na etapa 6.
- Para caber no Colab, trabalhe com um subconjunto. Sugestão: as 10 classes mais frequentes do nível escolhido, com no máximo 1.000 imagens por classe para treino. Mais dado é permitido, mas não é exigido.

Ponto de partida para carregar (ajuste os caminhos):

```
import pandas as pd, numpy as np, os
from PIL import Image

df = pd.read_csv("styles.csv", on_bad_lines="skip")
df["caminho"] = df["id"].astype(str).apply(lambda i: f"images/{i}.jpg")
df = df[df["caminho"].apply(os.path.exists)]          # fica só quem tem imagem

def carregar(caminho, tamanho=(60, 80)):
    return np.asarray(Image.open(caminho).convert("RGB").resize(tamanho))
```

3. O que o grupo deve fazer

Etapa 1. Conhecer o dado (EDA)

- Quantos produtos sobraram depois do cruzamento CSV x imagens.
- Distribuição de `masterCategory`, `subCategory` e `articleType`: quantas classes, quantos exemplos nas 10 maiores, quantas classes têm menos de 50 exemplos.
- Uma grade com 30 imagens e seus rótulos, para mostrar o que o modelo vai ver.
- **Decisão 1:** em qual nível o grupo vai trabalhar (`masterCategory`, `subCategory` ou `articleType`) e quais classes entram. Justifique com os números da distribuição e com o custo do erro para o negócio.

Etapa 2. Separar treino, validação e teste

- Divisão estratificada por classe. O teste fica intocado até a etapa 5.
- Diga os tamanhos dos três conjuntos.

Etapa 3. Baseline sem rede neural

- Um modelo simples que o time de negócio entenderia: por exemplo, regressão logística sobre os pixels reduzidos, ou sobre a média de cor da imagem, ou k-vizinhos.
- Acurácia na validação. Este é o número que qualquer rede precisa superar para se justificar.

Etapa 4. CNN do zero

- Uma CNN como a do notebook 02 (convolução, pooling, flatten, densa, softmax), adaptada ao tamanho da imagem e ao número de classes.
- Treine com `ModelCheckpoint` guardando a melhor época de validação. Mostre a curva de perda e acurácia (treino e validação) e diga em que época começou o overfitting, se começou.

Etapa 5. Transfer learning

- Reaproveite uma rede pré-treinada na ImageNet (MobileNetV2, como no notebook 04, ou outra) de dois jeitos:
  - **congelada**: só a cabeça nova treina;
  - **com fine-tuning**: descongele as últimas camadas com learning rate baixo.
- Mesmo conjunto de treino, validação e teste das etapas anteriores.
- Avalie os quatro modelos (baseline, CNN do zero, congelada, fine-tuning) **no teste, uma vez só**, e monte a tabela:

Modelo	Imagens de treino	Pesos treinados	Acurácia no teste	Tempo de treino

- Reduza o treino para 50 imagens por classe e repita só a CNN do zero e a rede congelada. O que acontece com cada uma?

Etapa 6. Do número à decisão de negócio

- Matriz de confusão do melhor modelo no teste. Quais pares de classes se confundem? Faz sentido para quem conhece o catálogo?
- Acurácia por classe. Em quais classes o modelo pode cadastrar sozinho e em quais o produto deve ir para revisão humana? Proponha uma regra concreta (por classe, por confiança da softmax, ou as duas).
- Estime o efeito: com a regra proposta, que fração dos produtos ainda passaria por uma pessoa?
- Discuta o que mudaria com fotos reais de fornecedores (fundo, iluminação, ângulo, produtos novos) e como o modelo seria monitorado depois de entrar em operação.

4. O que entregar

1. **Notebook** (`.ipynb`) executado, com as saídas visíveis, que rode no Colab do começo ao fim. Código comentado no estilo dos notebooks da aula: uma ideia por célula, sem classes.
2. **Relatório** de até 6 páginas ou 10 slides, para um gerente de e-commerce que não programa, com: o problema, a decisão 1 e sua justificativa, a tabela da etapa 5, a matriz de confusão, a regra de revisão humana e os riscos. Os números do relatório têm que ser os números do notebook.
3. **Apresentação** de 5 minutos na aula 3, uma pessoa do grupo, começando pela regra de revisão humana e não pelo modelo.

5. Critérios de avaliação

Critério	Peso	O que será observado
Rigor do experimento	30%	teste intocado até o fim; mesma divisão para todos os modelos; curvas de treino lidas corretamente
Comparação completa	25%	os quatro modelos na tabela, com o experimento de 50 imagens por classe
Leitura de negócio	30%	regra de revisão humana concreta e defendida pela matriz de confusão; riscos reais, não genéricos
Clareza	15%	notebook reprodutível e relatório que um gerente entende sem abrir o código

Não conta ponto: acurácia alta por si só. Um grupo com 85% e uma regra de revisão bem defendida vale mais que um grupo com 92% e nenhuma decisão.

6. Regras

- Qualquer biblioteca é permitida, desde que o grupo explique o que ela faz.
- Pode usar modelos pré-treinados de qualquer fonte; cite a origem e a licença.
- Assistentes de código são permitidos; o grupo responde por cada linha. Na apresentação, qualquer integrante pode ser perguntado sobre qualquer célula.
- O dataset é para uso educacional. Verifique os termos de uso na página do Kaggle antes de redistribuir imagens.

7. Dicas

- Comece pelo `masterCategory`. Só suba para `subCategory` ou `articleType` depois de ter o pipeline inteiro rodando com um resultado.
- Fixe as sementes aleatórias (`numpy`, `torch` ou `tensorflow`, e o `random_state` da divisão). Resultados que mudam a cada execução não são comparáveis.

- Guarde o tempo de treino de cada modelo com `time.time()`; ele entra na tabela.
- Se o Colab derrubar a sessão, reduza o número de imagens por classe antes de reduzir o número de épocas.
- Para transfer learning, use a normalização da rede pré-treinada (no caso da MobileNetV2, valores entre -1 e 1), não a sua.